PROGRAMAÇÃO WEB

Manual de Projeto

API REST — Sistema de Blog em Node.js

Docentes:
Célia Soares (P) Email: celiasoares@estg.ipvc.pt
Marcelo Antunes Fernandes (Docente Responsável - T) Email: fernandesmarcelo@estg.ipvc.pt

Objetivos do Projeto

Neste projeto, vais construir uma API REST completa para um sistema de blog. No final, serás capaz de:

- Configurar e estruturar um projeto Node.js profissional
- Criar um servidor Express com rotas organizadas
- Ligar e interagir com uma base de dados
- Implementar autenticação e autorização com JWT
- Desenvolver endpoints CRUD completos com boas práticas
- Validar dados e tratar erros de forma centralizada
- Testar a API com Postman

> **i Nota**
> Este manual acompanha-te passo a passo. Cada fase tem código completo que deves compreender, adaptar e expandir — não copiar cegamente.

Tecnologias Utilizadas

Tecnologia	Versão	Finalidade
`Node.js`	`>= 18.x`	Ambiente de execução
`Express`	`^4.x`	Framework web / servidor
`Mongoose`	`^8.x`	ODM para MongoDB
`jsonwebtoken`	`^9.x`	Autenticação JWT
`bcryptjs`	`^2.x`	Hash de passwords

Tecnologia	Versão	Finalidade
`express-validator`	`^7.x`	Validação de inputs
`dotenv`	`^16.x`	Variáveis de ambiente
`nodemon`	`^3.x`	Reload automático em dev

FASE 1

Configuração e Estrutura do Projeto

Semana 1 | Objetivo: servidor a correr em localhost

1.1 Inicializar o Projeto

Cria uma pasta para o projeto e inicializa o Node.js:

```
mkdir api-blog
cd api-blog
npm init -y
```

Instala as dependências de produção:

```
npm install express mongoose jsonwebtoken bcryptjs express-validator dotenv
```

Instala as dependências de desenvolvimento:

```
npm install --save-dev nodemon
```

Adiciona os seguintes scripts ao package.json:

```
"scripts": {
  "start": "node src/app.js",
  "dev":   "nodemon src/app.js"
}
```

1.2 Estrutura de Pastas

Cria a seguinte estrutura de diretórios:

```
api-blog/
├── src/
│   ├── config/
│   │   └── db.js            # Ligação à base de dados
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── artigoController.js
│   │   └── comentarioController.js
│   ├── middleware/
│   │   ├── auth.js          # Verificação do JWT
│   │   └── errorHandler.js  # Tratamento global de erros
```

```
│   ├── models/
│   │   ├── Utilizador.js
│   │   ├── Artigo.js
│   │   └── Comentario.js
│   ├── routes/
│   │   ├── auth.js
│   │   ├── artigos.js
│   │   └── comentarios.js
│   └── app.js              # Ponto de entrada
├── .env
├── .gitignore
└── package.json
```

✓ Dica

Nunca envies a pasta node_modules nem o ficheiro .env. Adiciona-os ao .gitignore desde o início.

1.3 Ficheiro .env e .gitignore

Cria o ficheiro .env na raiz do projeto:

```
PORT=3000
MONGODB_URI=mongodb://localhost:27017/blog
JWT_SECRET=umSegredoMuitoLongoESeguro123!
JWT_EXPIRES_IN=7d
```

Cria o ficheiro .gitignore:

```
node_modules/
.env
*.log
```

1.4 Servidor Express — app.js

Cria o ficheiro src/app.js com a configuração base do servidor:

```
const express = require('express');
const dotenv  = require('dotenv');
const connectDB = require('./config/db');

dotenv.config();
connectDB();
```

```
const app = express();

// Middleware global
app.use(express.json());

// Rota de teste
app.get('/', (req, res) => {
  res.json({ mensagem: 'API Blog online!' });
});

// Rotas da API (adicionar nas fases seguintes)
// app.use('/api/auth',      require('./routes/auth'));
// app.use('/api/artigos',   require('./routes/artigos'));

// Middleware de erros (deve ser o último)
// app.use(require('./middleware/errorHandler'));

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Servidor a correr na porta ${PORT}`);
});
```

ℹ Nota

Deixa as rotas comentadas por agora. Vais descomentá-las à medida que as crias nas fases seguintes.

FASE 2

Base de Dados e Modelos

Semana 1| Objetivo: modelos definidos e ligados ao MongoDB

2.1 Ligação ao MongoDB

Cria o ficheiro src/config/db.js:

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGODB_URI);
    console.log(`MongoDB ligado: ${conn.connection.host}`);
  } catch (err) {
    console.error('Erro ao ligar ao MongoDB:', err.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

⚠ Atenção

Certifica-te de que o MongoDB está a correr localmente (mongod) ou usa uma ligação MongoDB Atlas no .env.

2.2 Modelo de Utilizador

Cria o ficheiro src/models/Utilizador.js:

```
const mongoose = require('mongoose');
const bcrypt   = require('bcryptjs');

const utilizadorSchema = new mongoose.Schema({
  nome:  { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true,
           lowercase: true, trim: true },
  password: { type: String, required: true, minlength: 6,
              select: false },
  role: { type: String, enum: ['autor', 'admin'],
          default: 'autor' },
}, { timestamps: true });
```

```
// Hash automático antes de guardar
utilizadorSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

// Método de instância para comparar passwords
utilizadorSchema.methods.compararPassword = async function (pw) {
  return bcrypt.compare(pw, this.password);
};

module.exports = mongoose.model('Utilizador', utilizadorSchema);
```

ℹ Nota

O campo password tem select: false, ou seja, nunca é devolvido nas queries por defeito. Isto protege dados sensíveis automaticamente.

2.3 Modelo de Artigo

Cria o ficheiro src/models/Artigo.js:

```
const mongoose = require('mongoose');

const artigoSchema = new mongoose.Schema({
  titulo:    { type: String, required: true, trim: true },
  conteudo:  { type: String, required: true },
  categoria: { type: String, required: true,
               enum: ['tecnologia','ciência','desporto','cultura','outro'] },
  autor: {
    type: mongoose.Schema.Types.ObjectId,
    ref:  'Utilizador',
    required: true
  },
  publicado: { type: Boolean, default: false },
  imagem:    { type: String },
}, { timestamps: true });

module.exports = mongoose.model('Artigo', artigoSchema);
```

2.4 Modelo de Comentário

Cria o ficheiro src/models/Comentario.js:

```
const mongoose = require('mongoose');

const comentarioSchema = new mongoose.Schema({
  conteudo: { type: String, required: true, trim: true },
  autor: {
    type: mongoose.Schema.Types.ObjectId,
    ref:  'Utilizador',
    required: true
  },
  artigo: {
    type: mongoose.Schema.Types.ObjectId,
    ref:  'Artigo',
    required: true
  },
}, { timestamps: true });

module.exports = mongoose.model('Comentario', comentarioSchema);
```

✓ Dica

O campo ref: 'Artigo' e ref: 'Utilizador' permite usar .populate() para preencher os dados relacionados automaticamente nas queries.

FASE 3

Autenticação com JWT

Semana 2 | Objetivo: registo, login e proteção de rotas

3.1 Middleware de Autenticação

Cria o ficheiro src/middleware/auth.js:

```
const jwt          = require('jsonwebtoken');
const Utilizador   = require('../models/Utilizador');

// Verifica se o token JWT é válido
exports.proteger = async (req, res, next) => {
  let token;
  if (req.headers.authorization &&
      req.headers.authorization.startsWith('Bearer ')) {
    token = req.headers.authorization.split(' ')[1];
  }
  if (!token) {
    return res.status(401).json({ erro: 'Não autenticado' });
  }
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.utilizador = await Utilizador.findById(decoded.id);
    next();
  } catch {
    return res.status(401).json({ erro: 'Token inválido' });
  }
};

// Restringe acesso por role
exports.autorizar = (...roles) => (req, res, next) => {
  if (!roles.includes(req.utilizador.role)) {
    return res.status(403).json(
      { erro: 'Sem permissão para esta ação' }
    );
  }
  next();
};
```

3.2 Controller de Autenticação

Cria o ficheiro src/controllers/authController.js:

```
const jwt        = require('jsonwebtoken');
const Utilizador = require('../models/Utilizador');

// Gera um token JWT
const gerarToken = (id) =>
  jwt.sign({ id }, process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRES_IN });

// POST /api/auth/register
exports.register = async (req, res, next) => {
  try {
    const { nome, email, password } = req.body;
    const utilizador = await Utilizador.create(
      { nome, email, password }
    );
    const token = gerarToken(utilizador._id);
    res.status(201).json({ token, utilizador: {
      id: utilizador._id, nome: utilizador.nome,
      email: utilizador.email, role: utilizador.role
    }});
  } catch (err) { next(err); }
};

// POST /api/auth/login
exports.login = async (req, res, next) => {
  try {
    const { email, password } = req.body;
    // Buscar utilizador e incluir o campo password
    const utilizador = await Utilizador
      .findOne({ email }).select('+password');
    if (!utilizador ||
        !(await utilizador.compararPassword(password))) {
      return res.status(401).json(
        { erro: 'Email ou password incorretos' }
      );
    }
    const token = gerarToken(utilizador._id);
    res.json({ token });
  } catch (err) { next(err); }
};

// GET /api/auth/me
exports.getMe = async (req, res) => {
  res.json({ utilizador: req.utilizador });
};
```

3.3 Rotas de Autenticação

Cria o ficheiro src/routes/auth.js:

```
const express = require('express');
const { body } = require('express-validator');
const ctrl    = require('../controllers/authController');
const { proteger } = require('../middleware/auth');
const validar = require('../middleware/validar');

const router = express.Router();

router.post('/register',
  [
    body('nome').notEmpty().withMessage('Nome obrigatório'),
    body('email').isEmail().withMessage('Email inválido'),
    body('password').isLength({ min: 6 })
      .withMessage('Password: mínimo 6 caracteres'),
  ],
  validar,
  ctrl.register
);

router.post('/login',
  [
    body('email').isEmail(),
    body('password').notEmpty(),
  ],
  validar,
  ctrl.login
);

router.get('/me', proteger, ctrl.getMe);

module.exports = router;
```

3.4 Endpoints de Autenticação

Método	Rota	Auth?	Descrição
POST	/api/auth/register	Não	Cria um novo utilizador
POST	/api/auth/login	Não	Devolve token JWT

Método	Rota	Auth?	Descrição
GET	/api/auth/me	Sim	Perfil do utilizador atual

i Nota

Guarda o token JWT devolvido pelo login — vais incluí-lo no header de todas as rotas protegidas: Authorization: Bearer <token>

FASE 4

CRUD de Artigos e Comentários

Semana 2 | Objetivo: API completa e funcional

4.1 Controller de Artigos

Cria o ficheiro src/controllers/artigoController.js:

```
const Artigo = require('../models/Artigo');

// GET /api/artigos  — lista com paginação e filtros
exports.listar = async (req, res, next) => {
  try {
    const { categoria, page = 1, limit = 10 } = req.query;
    const filtro = { publicado: true };
    if (categoria) filtro.categoria = categoria;

    const total   = await Artigo.countDocuments(filtro);
    const artigos = await Artigo.find(filtro)
      .populate('autor', 'nome email')
      .sort({ createdAt: -1 })
      .skip((page - 1) * limit)
      .limit(Number(limit));

    res.json({
      total, page: Number(page),
      totalPaginas: Math.ceil(total / limit),
      artigos
    });
  } catch (err) { next(err); }
};

// GET /api/artigos/:id
exports.obter = async (req, res, next) => {
  try {
    const artigo = await Artigo.findById(req.params.id)
      .populate('autor', 'nome email');
    if (!artigo) return res.status(404)
      .json({ erro: 'Artigo não encontrado' });
    res.json({ artigo });
  } catch (err) { next(err); }
};

// POST /api/artigos
```

```
exports.criar = async (req, res, next) => {
  try {
    const artigo = await Artigo.create({
      ...req.body,
      autor: req.utilizador._id
    });
    res.status(201).json({ artigo });
  } catch (err) { next(err); }
};

// PUT /api/artigos/:id
exports.atualizar = async (req, res, next) => {
  try {
    const artigo = await Artigo.findById(req.params.id);
    if (!artigo) return res.status(404)
      .json({ erro: 'Artigo não encontrado' });
    // Só o próprio autor ou um admin pode editar
    if (artigo.autor.toString() !== req.utilizador._id.toString()
        && req.utilizador.role !== 'admin') {
      return res.status(403).json({ erro: 'Sem permissão' });
    }
    const atualizado = await Artigo.findByIdAndUpdate(
      req.params.id, req.body,
      { new: true, runValidators: true }
    );
    res.json({ artigo: atualizado });
  } catch (err) { next(err); }
};

// DELETE /api/artigos/:id
exports.apagar = async (req, res, next) => {
  try {
    const artigo = await Artigo.findById(req.params.id);
    if (!artigo) return res.status(404)
      .json({ erro: 'Artigo não encontrado' });
    if (artigo.autor.toString() !== req.utilizador._id.toString()
        && req.utilizador.role !== 'admin') {
      return res.status(403).json({ erro: 'Sem permissão' });
    }
    await artigo.deleteOne();
    res.status(204).send();
  } catch (err) { next(err); }
};
```

4.2 Rotas de Artigos

Cria o ficheiro src/routes/artigos.js:

```
const express = require('express');
const { body } = require('express-validator');
const ctrl    = require('../controllers/artigoController');
const { proteger } = require('../middleware/auth');
const validar = require('../middleware/validar');

const router = express.Router();

const validacaoArtigo = [
  body('titulo').notEmpty().withMessage('Título obrigatório'),
  body('conteudo').notEmpty().withMessage('Conteúdo obrigatório'),
  body('categoria')
    .isIn(['tecnologia','ciência','desporto','cultura','outro'])
    .withMessage('Categoria inválida'),
];

router.get('/',    ctrl.listar);
router.get('/:id', ctrl.obter);
router.post('/',   proteger, validacaoArtigo, validar, ctrl.criar);
router.put('/:id', proteger, validacaoArtigo, validar, ctrl.atualizar);
router.delete('/:id', proteger, ctrl.apagar);

module.exports = router;
```

4.3 Controller e Rotas de Comentários

Cria o ficheiro src/controllers/comentarioController.js:

```
const Comentario = require('../models/Comentario');
const Artigo     = require('../models/Artigo');

// GET /api/artigos/:artigoId/comentarios
exports.listar = async (req, res, next) => {
  try {
    const comentarios = await Comentario
      .find({ artigo: req.params.artigoId })
      .populate('autor', 'nome')
      .sort({ createdAt: -1 });
    res.json({ comentarios });
  } catch (err) { next(err); }
};

// POST /api/artigos/:artigoId/comentarios
```

```
exports.criar = async (req, res, next) => {
  try {
    const artigo = await Artigo.findById(req.params.artigoId);
    if (!artigo) return res.status(404)
      .json({ erro: 'Artigo não encontrado' });
    const comentario = await Comentario.create({
      conteudo: req.body.conteudo,
      autor:  req.utilizador._id,
      artigo: req.params.artigoId
    });
    res.status(201).json({ comentario });
  } catch (err) { next(err); }
};

// DELETE /api/comentarios/:id
exports.apagar = async (req, res, next) => {
  try {
    const comentario = await Comentario.findById(req.params.id);
    if (!comentario) return res.status(404)
      .json({ erro: 'Comentário não encontrado' });
    if (comentario.autor.toString() !==
        req.utilizador._id.toString()) {
      return res.status(403).json({ erro: 'Sem permissão' });
    }
    await comentario.deleteOne();
    res.status(204).send();
  } catch (err) { next(err); }
};
```

Cria o ficheiro src/routes/comentarios.js:

```
const express = require('express');
const { body } = require('express-validator');
const ctrl    = require('../controllers/comentarioController');
const { proteger } = require('../middleware/auth');
const validar = require('../middleware/validar');

const router = express.Router({ mergeParams: true });

router.get('/',  ctrl.listar);
router.post('/', proteger,
  [body('conteudo').notEmpty().withMessage('Conteúdo obrigatório')],
  validar, ctrl.criar
);
router.delete('/:id', proteger, ctrl.apagar);
```

```
module.exports = router;
```

4.4 Tabela de Endpoints Completa

Método	Rota	Auth?	Descrição
GET	/api/artigos	Não	Lista artigos (paginação + filtro)
GET	/api/artigos/:id	Não	Obtém um artigo
POST	/api/artigos	Sim	Cria um artigo
PUT	/api/artigos/:id	Sim	Atualiza um artigo (autor/admin)
DELETE	/api/artigos/:id	Sim	Apaga um artigo (autor/admin)
GET	/api/artigos/:id/comentarios	Não	Lista comentários de um artigo
POST	/api/artigos/:id/comentarios	Sim	Adiciona comentário a um artigo
DELETE	/api/comentarios/:id	Sim	Apaga comentário próprio

i Nota

Atualiza o app.js para incluir as rotas: descomenta as linhas app.use() que deixaste no passo 1.4 e adiciona a rota de comentários.

FASE 5

Validação, Erros e Testes

Semana 3 | Objetivo: API robusta e documentada

5.1 Middleware de Validação

Cria o ficheiro src/middleware/validar.js:

```
const { validationResult } = require('express-validator');

const validar = (req, res, next) => {
  const erros = validationResult(req);
  if (!erros.isEmpty()) {
    return res.status(400).json({ erros: erros.array() });
  }
  next();
};

module.exports = validar;
```

5.2 Middleware de Tratamento de Erros Global

Cria o ficheiro src/middleware/errorHandler.js:

```
const errorHandler = (err, req, res, next) => {
  console.error(err.stack);

  // Erro de ID inválido do Mongoose
  if (err.name === 'CastError') {
    return res.status(400).json({ erro: 'ID inválido' });
  }

  // Erro de duplicado (ex: email já existe)
  if (err.code === 11000) {
    const campo = Object.keys(err.keyValue)[0];
    return res.status(400).json(
      { erro: `${campo} já está em uso` }
    );
  }

  // Erro de validação do Mongoose
  if (err.name === 'ValidationError') {
```

```
    const msgs = Object.values(err.errors).map(e => e.message);
    return res.status(400).json({ erro: msgs.join(', ') });
  }

  // Erro genérico
  res.status(err.statusCode || 500).json({
    erro: err.message || 'Erro interno do servidor'
  });
};

module.exports = errorHandler;
```

Descomenta a linha do errorHandler no app.js — deve ser o último middleware registado:

```
app.use(require('./middleware/errorHandler'));
```

> **⚠ Atenção**
> O middleware de erros DEVE ter exatamente 4 parâmetros (err, req, res, next). Se tiver apenas 3, o Express não o reconhece como handler de erros.

5.3 Tabela de Códigos HTTP

Código	Nome	Quando usar
200	OK	Leitura ou atualização com sucesso
201	Created	Recurso criado com sucesso (POST)
204	No Content	Eliminação com sucesso (sem body)
400	Bad Request	Dados inválidos ou em falta
401	Unauthorized	Não autenticado (sem token válido)
403	Forbidden	Autenticado mas sem permissão
404	Not Found	Recurso não encontrado
409	Conflict	Recurso já existe (ex: email duplicado)
500	Internal Server Error	Erro inesperado no servidor

5.4 Testar com Postman — Fluxo Sugerido

Segue esta ordem para testar a API de forma sistemática:

1. Registar um utilizador (POST /api/auth/register)
2. Fazer login e copiar o token devolvido (POST /api/auth/login)

3. Configurar no Postman: Authorization > Bearer Token > colar o token
4. Criar um artigo (POST /api/artigos) — deve falhar sem token
5. Criar um artigo com o token — deve devolver 201
6. Listar artigos com filtro de categoria (GET /api/artigos?categoria=tecnologia)
7. Testar paginação (GET /api/artigos?page=1&limit=5)
8. Adicionar um comentário ao artigo criado
9. Tentar editar um artigo com um utilizador diferente — deve devolver 403
10. Apagar o comentário e o artigo criados

✓ Dica

Cria uma coleção Postman com todos os endpoints organizados por pasta (Auth, Artigos, Comentários). Esta coleção faz parte da entrega final.

5.5 app.js Final (completo)

O ficheiro app.js final deverá ter o seguinte aspeto:

```
const express    = require('express');
const dotenv     = require('dotenv');
const connectDB  = require('./config/db');

dotenv.config();
connectDB();

const app = express();
app.use(express.json());

// Rotas da API
app.use('/api/auth',      require('./routes/auth'));
app.use('/api/artigos',   require('./routes/artigos'));
app.use('/api/artigos',   require('./routes/comentarios'));
app.use('/api/comentarios', require('./routes/comentarios'));

// Rota não encontrada
app.use((req, res) => {
  res.status(404).json({ erro: 'Rota não encontrada' });
});

// Middleware global de erros
app.use(require('./middleware/errorHandler'));

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Servidor a correr na porta ${PORT}`);
```

```
});
```

Entregáveis

- Código-fonte no repositório Git (sem node_modules e sem .env)
- Ficheiro .env.example com as variáveis necessárias (sem valores reais)
- Coleção Postman exportada com todos os endpoints testados

✓ Dica

Bom trabalho! Lembra-te: o objetivo não é só fazer funcionar — é perceber porquê cada peça existe e como se relaciona com as outras.